



UNIVERSITY *of* WEST FLORIDA

Machine Learning PyTorch & Scikit Learn

Module 11: PyTorch Basics

Brian Jalaian, Ph.D.

Associate Professor

Intelligent Systems & Robotics Department

Table of Contents

Why do we need PyTorch?

What is PyTorch?

Basic Tensor Operations

Dataloaders

Torch-Based Neural Nets

Code Break

Computation Graph

Training with PyTorch

Custom Models

Weight Initialization

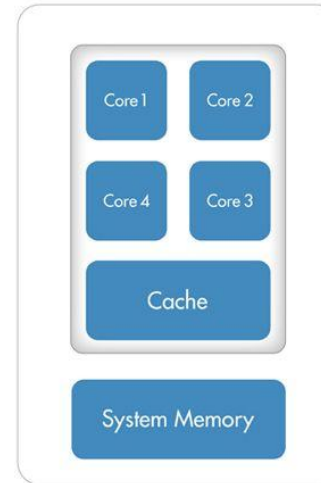
Saving Models

Code Break

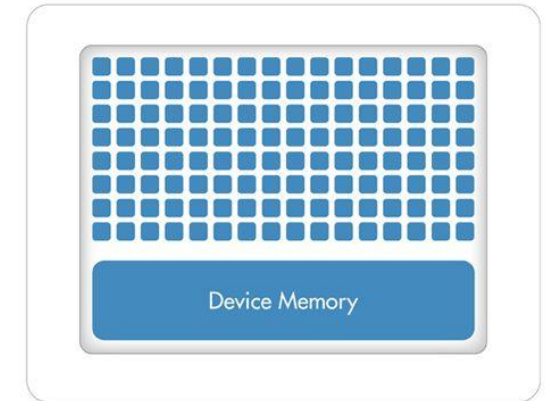
Why PyTorch?

- Previously, we used CPU to calculate our forward & backward passes
- Limited number of parallel computations on CPU...
- GPU can allow us to do large tensor operations in parallel, making large Neural Nets practical

CPU (Multiple Cores)



GPU (Hundreds of Cores)



High-Level Architectural Difference between
CPU and GPU

Specifications	Intel® Core™ i9-11900KB Processor	NVIDIA GeForce® RTX™ 3080 Ti
Base Clock Frequency	3.3 GHz	1.37 GHz
Cores	16 (32 threads)	10240
Memory Bandwidth	45.8 GB/s	912.1 GB/s
Floating-Point Calculations	742 GFLOPS	34.10 TFLOPS
Cost	~ \$540.00	~ \$1200.00

Comparison of a CPU and GPU

Why PyTorch?

- CUDA is NVIDIA's set of tools (150+ libraries) for directly instructing GPU, but can be low-level and complex
- PyTorch offers utilities for programming with tensor operations on GPU
- PyTorch offers useful classes & methods for constructing and training Neural Nets
- As of 2025: the default for research and industry applications

Standard C Code

```
void saxpy(int n, float a,
          float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}

int N = 1<<20;

// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
          float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

See how we make low-level calls in C to copy memory from location to location in CUDA

<https://blogs.nvidia.com/blog/what-is-cuda-2/>

What is PyTorch?

- Open Source Deep Learning tools
 - Associated w/ Meta
- GPU-accelerated tensor ops
- Dynamic Computation Graph
 - Used for calculating gradients in training
- Dataset Handling
 - DataLoader and Dataset classes to streamline training and testing
- A multimodal ecosystem
 - Complementary packages for specific data modalities like vision, audio, etc.

```
>>> class Model(nn.Module):
...     def __init__(self, input_size, hidden_size, output_size):
...         super().__init__()
...         self.layer1 = nn.Linear(input_size, hidden_size)
...         self.layer2 = nn.Linear(hidden_size, output_size)
...     def forward(self, x):
...         x = self.layer1(x)
...         x = nn.Sigmoid()(x)
...         x = self.layer2(x)
...         return x
>>> input_size = X_train_norm.shape[1]
>>> hidden_size = 16
>>> output_size = 3
>>> model = Model(input_size, hidden_size, output_size)

>>> for epoch in range(num_epochs):
...     for x_batch, y_batch in train_dl:
...         # 1. Generate predictions
...         pred = model(x_batch)[: , 0]
...         # 2. Calculate loss
...         loss = loss_fn(pred, y_batch)
...         # 3. Compute gradients
...         loss.backward()
...         # 4. Update parameters using gradients
...         optimizer.step()
...         # 5. Reset the gradients to zero
...         optimizer.zero_grad()
...     if epoch % log_epochs==0:
...         print(f'Epoch {epoch} Loss {loss.item():.4f}')
```

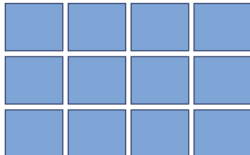
Example PyTorch model definition and training loop. Compare to our code from last week! Much less complex.

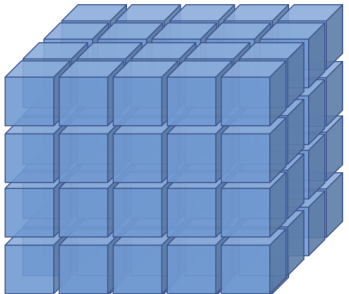
Basic Tensors

- Tensor: an algebraic structure in an n -dimensional space (rank)
 - Practically: a generalization of objects that includes scalar (rank 0 tensor), vectors (rank 1 tensor), matrices (rank 2 tensor) and so on
- We can use PyTorch to store and operate on these tensors
- Applications to Neural Nets and data handling
 - The Activations (output) of a layer could be a rank 1 tensor; $[a_i]$
 - A Neural Net layer could be a Rank 2 tensor; $[w_i][w_j]$
 - A Dataset could be a Rank 3 tensor; $[sample_id][input_features][label]$

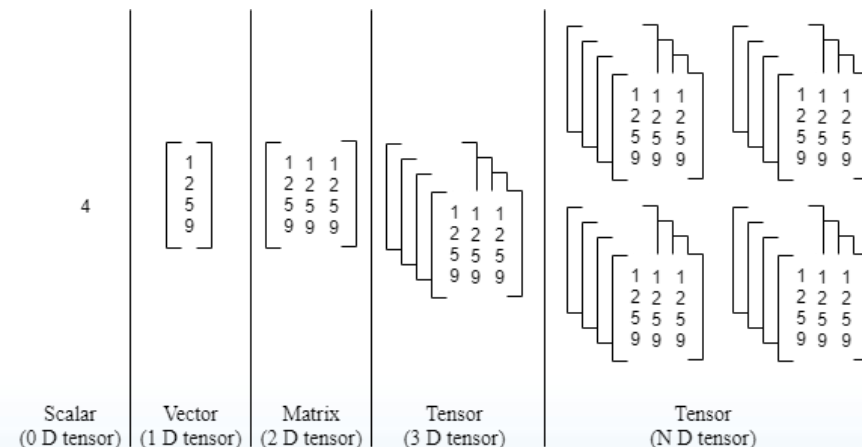
Rank 0: 
(scalar)

Rank 1: 
(vector)

Rank 2: (matrix)


Rank 3: 

Visualizations of tensors



PyTorch Tensor Operations



- Creating Tensors

- Create from list, Numpy array, or initialize using Torch

```
>>> a = [1, 2, 3]
>>> b = np.array([4, 5, 6], dtype=np.int32)
>>> t_a = torch.tensor(a)
>>> t_b = torch.from_numpy(b)
>>> print(t_a)
tensor([1, 2, 3])
>>> print(t_b)
tensor([4, 5, 6], dtype=torch.int32)
```

```
>>> rand_tensor = torch.rand(2,3)
>>> print(rand_tensor)
tensor([[0.1409, 0.2848, 0.8914],
        [0.9223, 0.2924, 0.7889]])
```

- Transpose

- Like the name suggests, take the transpose of the tensor between the two axes listed as arguments.

Creating Tensors

- Squeeze

- Remove an axis of size 1 (a redundant axis)
- The opposite operation: `unsqueeze()` adds a dimension of size 1

```
>>> t = torch.rand(3, 5)
>>> t_tr = torch.transpose(t, 0, 1)
>>> print(t.shape, '-->', t_tr.shape)
torch.Size([3, 5]) --> torch.Size([5, 3])
```

```
>>> t = torch.zeros(1, 2, 1, 4, 1)
>>> t_sqz = torch.squeeze(t, 2)
>>> print(t.shape, '-->', t_sqz.shape)
torch.Size([1, 2, 1, 4, 1]) --> torch.Size([1, 2, 4, 1])
```

Transpose of a Tensor

Torch Squeeze

- Concatenating Tensors

- Combine two tensors together along the same axis

```
>>> A = torch.ones(3)
>>> B = torch.zeros(2)
>>> C = torch.cat([A, B], axis=0)
>>> print(C)
tensor([1., 1., 1., 0., 0.])
```

Concatenation of Tensors

PyTorch Tensor Operations (continued)



- Stacking Tensors
 - Allocate a new axis and stack the two tensors together along it
- Splitting Tensors with `chunk()`
 - Divide a tensor along some axis and return its pieces divided equally into new tensors
- Multiplying Tensors (elementwise)
- Matrix Multiplication of Tensors
- Mean, Standard Deviation, and Sum
- *Research example: L2 Norm of Weight Magnitudes in an LLM*

```
>>> A = torch.ones(3)
>>> B = torch.zeros(3)
>>> S = torch.stack([A, B], axis=1)
>>> print(S)
tensor([[1., 0.],
        [1., 0.]])
```

Torch Stack

```
>>> t = torch.rand(6)
>>> print(t)
tensor([0.7576, 0.2793, 0.4031, 0.7347, 0.0293, 0.7999])
>>> t_splits = torch.chunk(t, 3)
>>> [item.numpy() for item in t_splits]
[array([0.758, 0.279], dtype=float32),
 array([0.403, 0.735], dtype=float32),
 array([0.029, 0.8 ], dtype=float32)]
```

Torch Split

```
>>> t3 = torch.multiply(t1, t2)
>>> print(t3)
tensor([[ 0.4426, -0.3114],
        [ 0.0660, -0.5970],
        [ 1.1249,  0.0150],
        [ 0.1569,  0.7107],
        [-0.0451, -0.0352]])
```

Torch Elementwise Multiplication

```
>>> t5 = torch.matmul(t1, torch.transpose(t2, 0, 1))
>>> print(t5)
tensor([[ 0.1312,  0.3860, -0.6267, -1.0096, -0.2943],
        [ 0.1647, -0.5310,  0.2434,  0.8035,  0.1980],
        [-0.3855, -0.4422,  1.1399,  1.5558,  0.4781],
```

Torch Matrix Multiplication

```
>>> t4 = torch.mean(t1, axis=0)
>>> print(t4)
tensor([-0.1373,  0.2028])
```

Torch Mean

```
>>> norm_t1 = torch.linalg.norm(t1, ord=2, dim=1)
>>> print(norm_t1)
tensor([0.6785, 0.5078, 1.1162, 0.5488, 0.1853])
```

Torch Norm

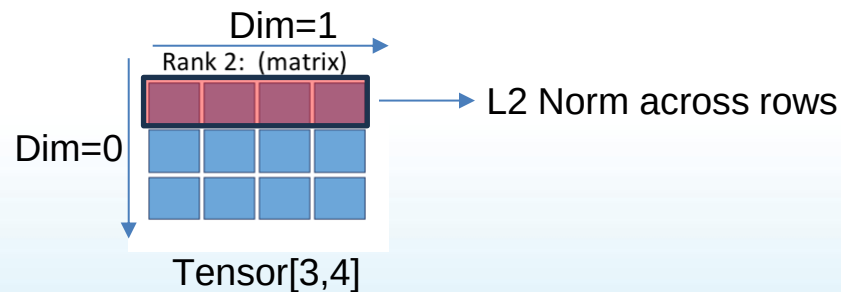
Research Example: L2 Norm of Tensors

```
with torch.no_grad():
    for i, layer in enumerate(model.model.layers):
        # Access the layers
        gate_proj = layer.mlp.gate_proj
        up_proj = layer.mlp.up_proj
        down_proj = layer.mlp.down_proj

        # Compute L2 norms for gate_proj and up_proj on CPU - save gpu
        gate_norm = torch.norm(gate_proj.weight.cpu(), dim=1)
        up_norm = torch.norm(up_proj.weight.cpu(), dim=1)
```

- From our lab, we compute L2 Norm of the weight magnitudes across the rows as a part of a metric for another process.
- Note the default of torch.norm is to take the Euclidean L2 Norm of the provided tensor (calculated indexing on columns; dim=1). The effect is L2Norm across rows

I highly recommend pen & paper or whiteboard for designing tensor ops!



Tensor Operations: Slice

- Probably the most critical – I find I use this most often in research
- Call the tensor object with
 - `tensor[start:stop:step, start:stop:step]`
- Examples:
 - `tensor[0, :]` means examine row 0, and return all columns ‘:’ for row 0
 - `tensor[1:, :]` means consider rows 1 and up, and return all columns for these rows
- Model outputs are often large tensors (ex: LLMs) – to get the model’s output at a specific position in the sequence, a user must slice the tensor
- *Research example: slicing the output tensor of an LLM*

```
import torch
# Create a sample 2D tensor
tensor = torch.tensor([[1, 2, 3],
                       [4, 5, 6],
                       [7, 8, 9]])

print("Original Tensor:")
print(tensor)

# Slice: Extract the first row
first_row = tensor[0, :]
print("\nFirst row:", first_row)

# Slice: Extract the first column
first_col = tensor[:, 0]
print("\nFirst column:", first_col)

# Slice: Extract a submatrix (bottom-right 2x2)
submatrix = tensor[1:, 1:]
print("\nBottom-right 2x2 submatrix:")
print(submatrix)

# Slice: Extract every other row
every_other_row = tensor[::2, :]
print("\nEvery other row:")
print(every_other_row)
```

Slice examples

```
Original Tensor:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])

First row: tensor([1, 2, 3])

First column: tensor([1, 4, 7])

Bottom-right 2x2 submatrix:
tensor([[5, 6],
        [8, 9]])

Every other row:
tensor([[1, 2, 3],
        [7, 8, 9]])
```

Research Example: Slice

```
for i, batch in enumerate(train_dataloader):
    with accelerator.accumulate(model):
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]
        labels = batch["labels"].squeeze(-1)

        outputs = model(input_ids, attention_mask=attention_mask)
        del input_ids, attention_mask
        torch.cuda.empty_cache()

        logits = outputs.logits
        last_token_logits = logits[:, -1, :] # Shape: [batch_size, vocab_size]

        # Compute and scale loss
        loss = nn.CrossEntropyLoss()(last_token_logits, labels)
```

An example from our active research that uses slice

Challenge: LLMs are answering Multiple Choice questions with A,B,C,D, or E.

- The output of the LLM is a tensor that looks like [batch_size][sequence_length][vocab_size]
- We slice using [:, -1, :]
- The output of the slice is the probabilities of tokens at the final position in the sequence for each sample in the batch; this corresponds to the model's answer to the question
- We can then take a loss to the known label (A,B,C,D, or E) to train the model

DataLoaders



- Don't load entire dataset onto GPU – for large datasets and models, this will exceed the capacity of the GPU and cause a crash
- Send smaller *batches* of data onto GPU – like our mini-batches in SGD
- Torch DataLoader: a class to iterate through the dataset in these mini-batches.
- *Research Code: Dataloaders for LLMs*

```
>>> from torch.utils.data import DataLoader
```

```
>>> t = torch.arange(6, dtype=torch.float32)
```

Initialize a tensor t

```
>>> data_loader = DataLoader(t, batch_size=3, drop_last=False)
```

```
>>> for i, batch in enumerate(data_loader, 1):
```

```
...     print(f'batch {i}:', batch)
```

```
batch 1: tensor([0., 1., 2.])
```

```
batch 2: tensor([3., 4., 5.])
```

*Initialize a dataloader and then
iterate through it*

```
#load train set
def load_train_set(batch_size,tokenizer):
    dataset = load_dataset("commonsense_qa")

    combined_preprompted = dataset.map(
        lambda x: combine_prompt_CQA(x),
        batched=True,
        remove_columns=dataset["train"].column_names,
    )

    tokenized_train = combined_preprompted['train'].map(lambda x: tokenize_function(x,tokenizer), batched=True, remove_columns=["inputs", "labels"])
    data_collator = DataCollatorWithPadding(tokenizer=tokenizer, return_tensors="pt")
    train_dataloader = DataLoader(tokenized_train, batch_size=batch_size, collate_fn=data_collator)

    return train_dataloader
```

Lab's production code for creating the Training Dataloader for an LLM

PyTorch Neural Nets

- You can define your own model architecture using a sequence of torch.nn objects
- Every model needs at least the following classes implemented to comply with the Module interface:
 - `__init__()`
 - How to initialize the model
 - Architectural elements like layers are initialized here
 - `forward()`
 - How to evaluate the forward pass from input to output
 - Specify the sequence of transformations that happen to the input data

```
>>> class Model(nn.Module):
...     def __init__(self, input_size, hidden_size, output_size):
...         super().__init__()
...         self.layer1 = nn.Linear(input_size, hidden_size)
...         self.layer2 = nn.Linear(hidden_size, output_size)
...     def forward(self, x):
...         x = self.layer1(x)
...         x = nn.Sigmoid()(x)
...         x = self.layer2(x)
...         return x
>>> input_size = X_train_norm.shape[1]
>>> hidden_size = 16
>>> output_size = 3
>>> model = Model(input_size, hidden_size, output_size)
>>> learning_rate = 0.001
>>> loss_fn = nn.CrossEntropyLoss()
>>> optimizer = torch.optim.Adam(model.parameters(),
lr=learning_rate)
>>> num_epochs = 100
>>> loss_hist = [0] * num_epochs
>>> accuracy_hist = [0] * num_epochs
>>> for epoch in range(num_epochs):
...     for x_batch, y_batch in train_dl:
...         pred = model(x_batch)
...         loss = loss_fn(pred, y_batch)
...         loss.backward()
...         optimizer.step()
...         optimizer.zero_grad()
...         loss_hist[epoch] += loss.item()*y_batch.size(0)
...         is_correct = (torch.argmax(pred, dim=1) == y_batch).float()
...         accuracy_hist[epoch] += is_correct.sum()
...     loss_hist[epoch] /= len(train_dl.dataset)
...     accuracy_hist[epoch] /= len(train_dl.dataset)
```

Torch nn Components: A List

- <https://pytorch.org/docs/stable/nn.html>
- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- **Non-linear Activations (weighted sum, nonlinearity) – what we will use as the activation for our layers (Sigmoid, and Relu)**
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- **Linear Layers – what we are using for fully connected neural net layers**
- Dropout Layers
- Sparse Layers
- Distance Functions
- **Loss Functions – what we will use for our loss function, especially Cross Entropy Loss and MSELoss**
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization
 - Aliases

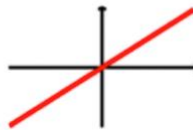
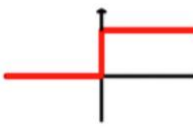
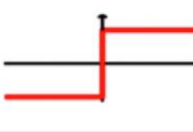
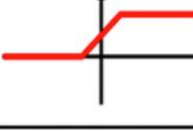
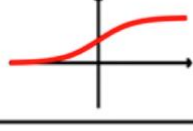
Don't be scared! There are a lot of great tools in your toolkit already!

The rest of this semester will be (roughly speaking) learning how to plug different components into a Neural Net pipeline to create diverse architectures

There is a lot of research to be done using these tools!

Torch nn Activation Functions

- Which activation function should you use?
- Advice: stick to Relu() to start for NNs, and branch out as you gain expertise

Activation function	Equation	Example	1D graph
Linear	$\sigma(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\sigma(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

Code Break: Chapter 12

Research Highlight: Torch-based MLP

```
class SmallMLP(nn.Module):
    def __init__(self, input_size=784, hidden_size=128, num_classes=10, num_layers=8):
        super(SmallMLP, self).__init__()

        layers = []
        layers = [nn.Flatten()] # Ensure input is flattened from [batch, 1, 28, 28] to [batch, 784]

        layers.append(nn.Linear(input_size, hidden_size)) # First Layer from 784 -> hidden_size
        layers.append(nn.ReLU())

        for _ in range(num_layers - 2): # Hidden Layers
            layers.append(nn.Linear(hidden_size, hidden_size))
            layers.append(nn.ReLU())
        layers.append(nn.Linear(hidden_size, num_classes)) # Output Layer

        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

- From active research in the lab; a toy MNIST neural net
- Note how we maintain a list of layers in layers[] and append nn components to it
- Note how we can use nn.Sequential to store the list of layers and call them later in forward()

Alternatives to Sequential

- We've already seen a model using Sequential...
- We can also specify a list of the modules as an instance of `nn.ModuleList()`
- Suppose in some architecture, there is some process I may want to partially skip – Module List is one way to cleanly implement these

```
class SkipModel(nn.Module):
    def __init__(self, in_features, hidden_dim, out_features):
        super(SkipModel, self).__init__()
        self.layers = nn.ModuleList([
            nn.Linear(in_features, hidden_dim), # First layer
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), # Second layer
            nn.ReLU(),
            nn.Linear(hidden_dim, out_features) # Output layer
        ])

    def forward(self, x):
        skip = x # Save input for skip connection
        for layer in self.layers:
            x = layer(x) # Sequentially apply layers

        return x + skip # Add skip connection before returning
```

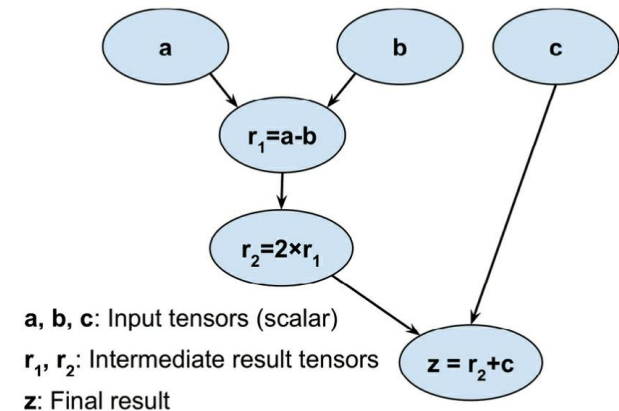
```
>>> class MyModule(nn.Module):
...     def __init__(self):
...         super().__init__()
...         l1 = nn.Linear(2, 4)
...         a1 = nn.ReLU()
...         l2 = nn.Linear(4, 4)
...         a2 = nn.ReLU()
...         l3 = nn.Linear(4, 1)
...         a3 = nn.Sigmoid()
...         l = [l1, a1, l2, a2, l3, a3]
...         self.module_list = nn.ModuleList(l)
...
...     def forward(self, x):
...         for f in self.module_list:
...             x = f(x)
...         return x
```

*Example of managing layers
with ModuleList*

PyTorch Computation Graph

- PyTorch constructs a *computation graph* during the forward call to map how inputs are transformed into outputs
- The backwards pass flows along this computation graph in the reverse direction from the arrows
- Previous gradients are used to calculate the next gradients; we saw the math underlying this last week with the chain rule
- Computation graph turns backprop of gradients into dynamic programming problem

Computation graph implementing the equation $z = 2 \times (a - b) + c$



An example of turning a function into an equivalent computation graph

PyTorch Computation Graph

- Good visualization tool: Torchviz

```
import torch
import torch.nn as nn
import torch.optim as optim
!pip install torchviz
from torchviz import make_dot

# Simple MLP model
class SimpleNet(nn.Module):
    def __init__(self):
        super(SimpleNet, self).__init__()
        self.fc1 = nn.Linear(2, 4)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(4, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

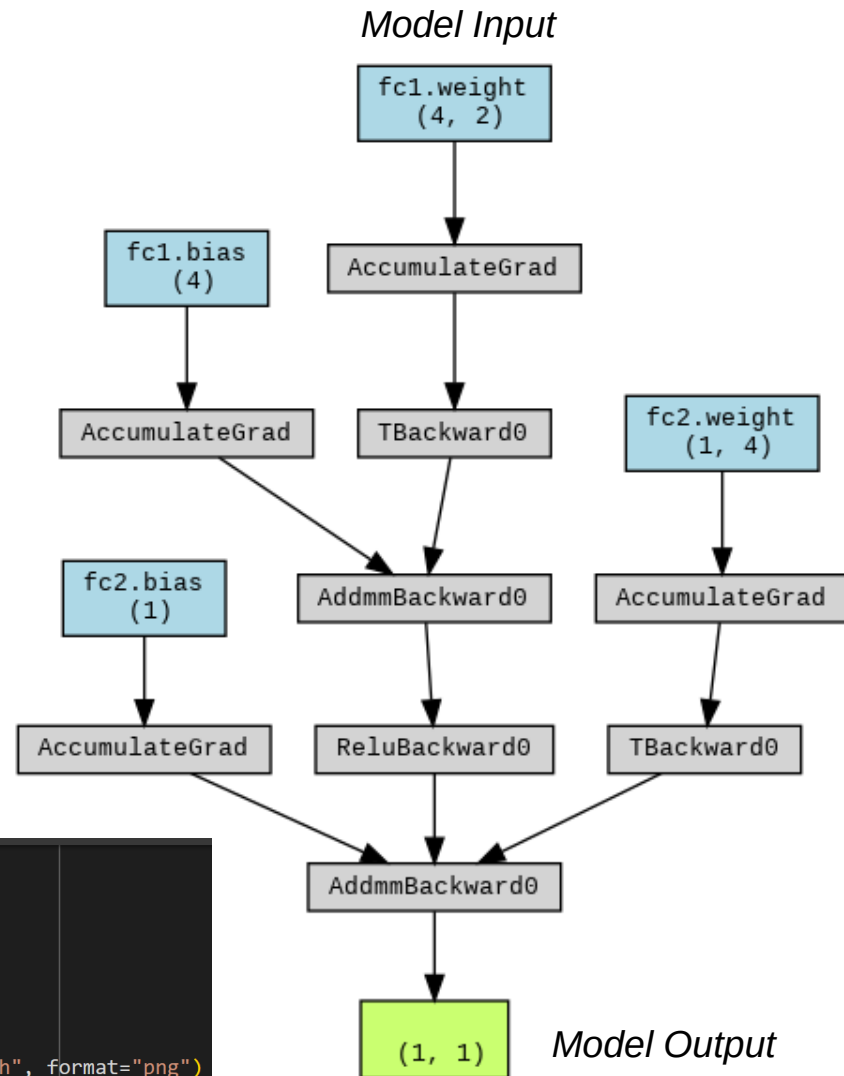
# Instantiate model
model = SimpleNet()

# Sample input tensor
x = torch.randn(1, 2)

# Forward pass
y = model(x)

# Visualize computation graph
make_dot(y, params=dict(model.named_parameters())).render("computation_graph", format="png")

# Display the graph in Colab
from IPython.display import Image
Image("computation_graph.png")
```



Computation Graph: Research Example

- My learning: in order to extract gradients and NOT break the graph dependencies, I have to explicitly refer to the memory addresses of the individual parameters using `id()`
- Gradients are linked to memory addresses of params – extracting the params with an assignment like `=` breaks the dependency between the extracted grads and params if I want to call `torch.autograd` later

```
#we have to calculate the top n eigenvectors of the 1st block; orthonormality of v is enforced per-block
for module_idx, (name, module) in enumerate(trainable_modules):
    computed_dim = 0 #iterator for tracking dimensions calculated for this block
    params_module = list(module.parameters())
    # Create a mapping from module parameters to their gradients
    param_to_grad = {id(param): grad for param, grad in zip(model.parameters(), grads)}
    # Extract only the gradients corresponding to the module
    grads_module = [param_to_grad[id(param)] for param in params_module if id(param) in param_to_grad]

    #debug params and gradients
    ...

    for i, g in enumerate(grads_module):
        print(f"Grad {i}: requires_grad={g.requires_grad if g is not None else None}, shape={g.shape if g is not None else None}")
    for i, p in enumerate(params_module):
        print(f"Param {i}: requires_grad={p.requires_grad}, shape={p.shape}")
    for i, (p, g) in enumerate(zip(params_module, grads_module)):
        if g is not None:
            dependency = torch.autograd.grad(g, p, grad_outputs=torch.ones_like(g), allow_unused=True, retain_graph=True)
            print(f"Grad {i} depends on Param {i}: {dependency is not None and dependency[0] is not None}")
        else:
            print(f"Grad {i} is None")
    ...
```

Weight Initialization

- Homogeneous, random initialization actually can perform badly in training
- If we vary the random distribution by layer, we can improve stability in training
- Two examples of methods for initialization schemes:
 - Xavier/Glorot Init
<http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>
 - Uniform
 - Gaussian
 - Lottery Ticket Rewind:
<https://arxiv.org/pdf/2003.02389>
 - Use an identical weight init to a previous training run on a modified model
- Depending on your use case, you may need a custom init scheme. Research!
- Init schemes are available layer-by-layer using `nn.init`

```
>>> class MyModule(nn.Module):  
...     def __init__(self):  
...         super().__init__()  
...         self.w1 = torch.empty(2, 3, requires_grad=True)  
...         nn.init.xavier_normal_(self.w1)  
...         self.w2 = torch.empty(1, 2, requires_grad=True)  
...         nn.init.xavier_normal_(self.w2)
```

$$\sigma = \frac{\sqrt{2}}{\sqrt{n_{in} + n_{out}}}$$

Gaussian Std Dev for Xavier Initialization parametrized by number of weights in the layer

Custom Modules/Layers

- You can write new layers as Modules
- These still require `__init__` and `forward`, at least
- Example on right:
 - Initialize a torch tensor for the weights
 - Make sure it will be added to computation graph using `nn.Parameter()`
 - Initialize it using Xavier-Uniform
 - Initialize bias vector as a torch tensor and likewise treat it as module parameter
 - When doing a forward pass in training, use torch to add noise to the input features
 - At inference time, do not add noise
 - Return the forward pass: matrix multiply input features and weights, then add bias

```
>>> class NoisyLinear(nn.Module):
...     def __init__(self, input_size, output_size,
...                   noise_stddev=0.1):
...         super().__init__()
...         w = torch.Tensor(input_size, output_size)
...         self.w = nn.Parameter(w) # nn.Parameter is a Tensor
...                                     # that's a module parameter.
...         nn.init.xavier_uniform_(self.w)
...         b = torch.Tensor(output_size).fill_(0)
...         self.b = nn.Parameter(b)
...         self.noise_stddev = noise_stddev
...
...     def forward(self, x, training=False):
...         if training:
...             noise = torch.normal(0.0, self.noise_stddev, x.shape)
...             x_new = torch.add(x, noise)
...         else:
...             x_new = x
...         return torch.add(torch.mm(x_new, self.w), self.b)
```

Example of a custom layer that adds noise to training data, hoping to improve generalization of the final model

Checkpoints: Saving An Experiment

- Saving a model is easy!
- Once you're done training, `torch.save()` will send the model weights to disk, for posterity
- Reload the model later with `torch.load()`

```
torch.save(model, "entire_model.pth")  
model_loaded = torch.load("entire_model.pth")  
model_loaded.eval()
```

Simple save and load

```
# Save model and optimizer  
torch.save({"model_state": model.state_dict(),  
           "optimizer_state": optimizer.state_dict()}, "checkpoint.pth")  
  
# Load model and optimizer  
checkpoint = torch.load("checkpoint.pth")  
model.load_state_dict(checkpoint["model_state"])  
optimizer.load_state_dict(checkpoint["optimizer_state"])
```

Save the optimizer as well if training later

Code Break: Ch 13